

如何基于Ubuntu 24.04部署原生K8S 1.30.0集群？

基于containerd容器运行时部署k8s 1.30.0集群

一、K8S集群主机准备

1.1 主机操作系统说明

序号	操作系统及版本	备注
1	Ubuntu 24.04	

1.2 主机硬件配置说明

需求	CPU	内存	硬盘	角色	主机名
值	8C	8G	1024GB	master	k8s-master01
值	8C	16G	1024GB	worker(node)	k8s-worker01
值	8C	16G	1024GB	worker(node)	k8s-worker02

1.3 主机配置

1.3.1 主机名配置

由于本次使用3台主机完成kubernetes集群部署，其中1台为master节点,名称为k8s-master01;其中2台为worker节点，名称分别为：k8s-worker01及k8s-worker02

```
1 master节点
2 # hostnamectl set-hostname k8s-master01
```

```
1 worker01节点
2 # hostnamectl set-hostname k8s-worker01
```

```
1 worker02节点
2 # hostnamectl set-hostname k8s-worker02
```

1.3.2 主机IP地址配置

```
1 k8s-master01节点IP地址为: 192.168.10.140/24
2 root@k8s-master01:~# vim /etc/netplan/01-network-manager-all.yaml
3 root@k8s-master01:~# cat /etc/netplan/01-network-manager-all.yaml
4 network:
5   version: 2
6   renderer: networkd
7   ethernets:
8     ens33:
9       dhcp4: no
10      addresses:
11        - 192.168.10.140/24
12      routes:
13        - to: default
14          via: 192.168.10.2
15      nameservers:
16        addresses: [119.29.29.29,114.114.114.114,8.8.8.8]
```

```
1 # netplan apply
```

```
1 k8s-worker01节点IP地址为: 192.168.10.141/24
2 root@k8s-worker01:~# vim /etc/netplan/01-network-manager-all.yaml
3 root@k8s-worker01:~# cat /etc/netplan/01-network-manager-all.yaml
4 network:
5   version: 2
6   renderer: networkd
7   ethernets:
8     ens33:
9       dhcp4: no
10      addresses:
11        - 192.168.10.141/24
12      routes:
13        - to: default
14          via: 192.168.10.2
15      nameservers:
16        addresses: [119.29.29.29,114.114.114.114,8.8.8.8]
```

```
1 # netplan apply
```

```
1 k8s-worker02节点IP地址为: 192.168.10.142/24
2 root@k8s-worker02:~# vim /etc/netplan/01-network-manager-all.yaml
3 root@k8s-worker02:~# cat /etc/netplan/01-network-manager-all.yaml
4 network:
5   version: 2
6   renderer: networkd
7   ethernets:
8     ens33:
9       dhcp4: no
10      addresses:
11        - 192.168.10.142/24
12      routes:
13        - to: default
14          via: 192.168.10.2
15      nameservers:
16        addresses: [119.29.29.29, 114.114.114.114, 8.8.8.8]
```

```
1 # netplan apply
```

1.3.3 主机名与IP地址解析

所有集群主机均需要进行配置。

```
1 # cat >> /etc/hosts << EOF
2 192.168.10.140 k8s-master01
3 192.168.10.141 k8s-worker01
4 192.168.10.142 k8s-worker02
5 EOF
```

1.3.4 时间同步配置

```
1 查看时间
2 # date
3 Thu Sep  7 05:39:21 AM UTC 2024
```

```
1 更换时区
2 # timedatectl set-timezone Asia/Shanghai
```

```
1 再次查看时间
2 # date
3 Thu Sep  7 01:39:51 PM CST 2024
```

```
1 安装ntpd命令
2 # apt install ntpdate
```

```
1 使用ntpd命令同步时间
2 # ntpdate time1.aliyun.com
```

```
1 通过计划任务实现时间同步
2
3 # crontab -e
4 no crontab for root - using an empty one
5
6 select an editor. To change later, run 'select-editor'.
7 1. /bin/nano          <---- easiest
8 2. /usr/bin/vim.basic
9 3. /usr/bin/vim.tiny
10 4. /bin/ed
11
12 Choose 1-4 [1]: 2
13
14 .....
15 0 */1 * * * ntpdate time1.aliyun.com
16
17
18
19 # crontab -l
20 .....
21 0 */1 * * * ntpdate time1.aliyun.com
```

1.3.5 配置内核转发及网桥过滤

所有主机均需要操作。

```
1 创建加载内核模块文件
2 # cat << EOF | tee /etc/modules-load.d/k8s.conf
3 overlay
4 br_netfilter
5 EOF
```

```
1 本次执行，手动加载此模块
2 # modprobe overlay
3 # modprobe br_netfilter
```

```
1 查看已加载的模块
2 # lsmod | egrep "overlay"
3 overlay                151552  0
4
5 # lsmod | egrep "br_netfilter"
6 br_netfilter            32768  0
7 bridge                  307200  1 br_netfilter
```

```
1 添加网桥过滤及内核转发配置文件
2 # cat << EOF | tee /etc/sysctl.d/k8s.conf
3 net.bridge.bridge-nf-call-ip6tables = 1
4 net.bridge.bridge-nf-call-iptables = 1
5 net.ipv4.ip_forward = 1
6 EOF
```

```
1 加载内核参数
2 # sysctl --system
```

1.3.6 安装ipset及ipvsadm

所有主机均需要操作。

```
1 安装ipset及ipvsadm
2 # apt install ipset ipvsadm
```

```
1 配置ipvsadm模块加载
2 添加需要加载的模块
3 # cat << EOF | tee /etc/modules-load.d/ipvs.conf
4 ip_vs
5 ip_vs_rr
6 ip_vs_wrr
7 ip_vs_sh
8 nf_conntrack
9 EOF
```

```
1 创建加载模块脚本文件
2 # cat << EOF | tee ipvs.sh
3 #!/bin/sh
4 modprobe -- ip_vs
5 modprobe -- ip_vs_rr
6 modprobe -- ip_vs_wrr
7 modprobe -- ip_vs_sh
8 modprobe -- nf_conntrack
9 EOF
```

```
1 执行脚本文件，加载模块
2 # sh ipvs.sh
```

1.3.7 关闭SWAP分区

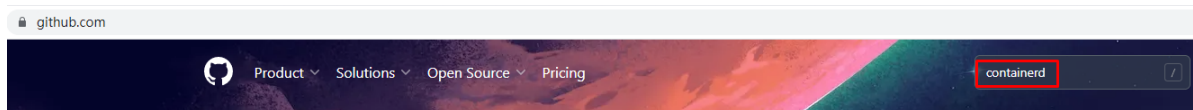
修改完成后需要重启操作系统，如不重启，可临时关闭，命令为swapoff -a

```
1 永远关闭swap分区，需要重启操作系统
2 # vim /etc/fstab
3 # cat /etc/fstab
4 .....
5
6 #/swap.img          none      swap      sw         0          0
7
8 在上一行中行首添加#
```

二、K8S集群容器运行时 Containerd准备

安装docker-ce方法：<https://docs.docker.com/engine/install/ubuntu/>

2.1 Containerd部署文件获取



 **containerd/containerd**

An open and reliable container runtime

kubernetes

containers

cncf

oci

docker

cri

hacktoberfest

containerd

☆ 13.6k

Go

Apache-2.0 license

Updated 8 hours ago

github.com/containerd/containerd

Product Solutions Open Source Pricing

Search or jump to...

containerd / containerd (Public)

Code Issues 433 Pull requests 248 Discussions Actions Projects 1 Security 13 Insights

main 10 branches 188 tags

Go to file Code

About

An open and reliable container runtime

containerd.io

docker kubernetes containers cncf

oci cri hacktoberfest containerd

Readme

Apache-2.0 license

Code of conduct

Security policy

Activity

14.6k stars

320 watching

3.1k forks

Report repository

Releases 179

containerd 1.7.3 (Latest)

3 weeks ago

+ 178 releases

fuweid Merge pull request #8930 from AkihiroSuda/rocky8.8	3688719 2 days ago	12,529 commits
.devcontainer	Adds a file header	5 months ago
.github	CI: update Rocky Linux to 8.8	last week
api	archive: use 1970-01-01 as the whiteout timestamp	2 months ago
archive	archive: use 1970-01-01 as the whiteout timestamp	2 months ago
cio	fix: allow attaching to any combination of stdin/stdout/stderr	2 weeks ago
cluster	Update DEPLOY_DIR and VERSION to match installation script	3 years ago
cmd	Update dependencies after protobuf update in hcsshim	last week
containers	go.mod: github.com/containerd/typeurl/v2 v2.1.0	6 months ago
content	Change to Readdirnames for some cases	3 months ago
contrib	update to go1.20.7, go1.19.12	2 weeks ago
defaults	Cleanup build constraints	8 months ago
diff	Revert "Revert 416899f"	last month
docs	delete checkout branch in doc	2 weeks ago
errdefs	Enable dupword linter	7 months ago

github.com/containerd/containerd/releases/tag/v1.7.3

containerd-1.7.3-linux-ppc64le.tar.gz.sha256sum	104 Bytes	3 weeks ago
containerd-1.7.3-linux-riscv64.tar.gz	33.9 MB	3 weeks ago
containerd-1.7.3-linux-riscv64.tar.gz.sha256sum	104 Bytes	3 weeks ago
containerd-1.7.3-linux-s390x.tar.gz	35.9 MB	3 weeks ago
containerd-1.7.3-linux-s390x.tar.gz.sha256sum	102 Bytes	3 weeks ago
containerd-1.7.3-windows-amd64.tar.gz	32.5 MB	3 weeks ago
containerd-1.7.3-windows-amd64.tar.gz.sha256sum	104 Bytes	3 weeks ago
containerd-static-1.7.3-linux-amd64.tar.gz	35 MB	3 weeks ago
containerd-static-1.7.3-linux-amd64.tar.gz.sha256sum	109 Bytes	3 weeks ago
containerd-static-1.7.3-linux-arm64.tar.gz	31.6 MB	3 weeks ago
containerd-static-1.7.3-linux-arm64.tar.gz.sha256sum	109 Bytes	3 weeks ago
containerd-static-1.7.3-linux-ppc64le.tar.gz	31.2 MB	3 weeks ago
containerd-static-1.7.3-linux-ppc64le.tar.gz.sha256sum	111 Bytes	3 weeks ago
containerd-static-1.7.3-linux-riscv64.tar.gz	32.8 MB	3 weeks ago
containerd-static-1.7.3-linux-riscv64.tar.gz.sha256sum	111 Bytes	3 weeks ago
containerd-static-1.7.3-linux-s390x.tar.gz	34.5 MB	3 weeks ago
containerd-static-1.7.3-linux-s390x.tar.gz.sha256sum	109 Bytes	3 weeks ago
cri-containerd-1.7.3-linux-amd64.tar.gz	99.3 MB	3 weeks ago
cri-containerd-1.7.3-linux-amd64.tar.gz.sha256sum	106 Bytes	3 weeks ago
cri-containerd-1.7.3-linux-arm64.tar.gz	88.7 MB	3 weeks ago
cri-containerd-1.7.3-linux-arm64.tar.gz.sha256sum	106 Bytes	3 weeks ago
cri-containerd-1.7.3-linux-ppc64le.tar.gz	87.9 MB	3 weeks ago
cri-containerd-1.7.3-linux-ppc64le.tar.gz.sha256sum	108 Bytes	3 weeks ago
cri-containerd-1.7.3-linux-riscv64.tar.gz	91.2 MB	3 weeks ago

- 1 下载指定版本containerd
- 2 # wget
https://github.com/containerd/containerd/releases/download/v1.7.16/cri-containerd-1.7.16-linux-amd64.tar.gz

- 1 解压安装
- 2 # tar xf cri-containerd-1.7.16-linux-amd64.tar.gz -C /

2.2 Containerd配置文件生成并修改

```
1 创建配置文件目录
2 # mkdir /etc/containerd
```

```
1 生成配置文件
2 # containerd config default > /etc/containerd/config.toml
```

```
1 修改第67行
2 # vim /etc/containerd/config.toml
3
4 sandbox_image = "registry.k8s.io/pause:3.9" 由3.8修改为3.9
```

```
1 如果使用阿里云容器镜像仓库，也可以修改为：
2 sandbox_image = "registry.aliyuncs.com/google_containers/pause:3.9" 由3.8修改为3.9
```

```
1 修改第139行
2 # vim /etc/containerd/config.toml
3
4 SystemdCgroup = true 由false修改为true
```

2.3 Containerd启动及开机自启动

```
1 设置开机自启动并现在启动
2 # systemctl enable --now containerd
```

```
1 验证其版本
2 # containerd --version
```

三、K8S集群部署

3.1 K8S集群软件apt源准备

本次使用kubernetes社区软件源仓库或阿里云软件源仓库

下载用于 Kubernetes 软件包仓库的公共签名密钥

所有仓库都使用相同的签名密钥，因此你可以忽略URL中的版本：

K8S社区

```
1 # curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.30/deb/Release.key | sudo
   gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg
```

阿里云

```
1 # curl -fsSL https://mirrors.aliyun.com/kubernetes-
   new/core/stable/v1.30/deb/Release.key |
2   gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg
```

添加 Kubernetes apt 仓库

请注意，此仓库仅包含适用于 Kubernetes 1.30 的软件包；对于其他 Kubernetes 次要版本，则需要更改 URL 中的 Kubernetes 次要版本以匹配你所需的次要版本。

此操作会覆盖 /etc/apt/sources.list.d/kubernetes.list 中现存的所有配置，如果有的情况下。

K8S社区

```
1 # echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg]
   https://pkgs.k8s.io/core:/stable:/v1.30/deb/ /' | sudo tee
   /etc/apt/sources.list.d/kubernetes.list
```

阿里云

```
1 # echo "deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg]
   https://mirrors.aliyun.com/kubernetes-new/core/stable/v1.30/deb/ /" |
2   tee /etc/apt/sources.list.d/kubernetes.list
```

更新 apt 包索引

```
1 # sudo apt-get update
```

3.2 K8S集群软件安装及kubelet配置

所有节点均可安装

3.2.1 k8s集群软件安装

```
1 查看软件列表
2 # apt-cache policy kubeadm
3 kubeadm:
4   Installed: (none)
5   Candidate: 1.30.0-1.1
6   Version table:
7     1.30.0-1.1 500
8     500 https://pkgs.k8s.io/core:/stable:/v1.30/deb Packages
```

```
1 查看软件列表及其依赖关系
2 # apt-cache showpkg kubeadm
3 Package: kubeadm
4 Versions:
5 1.30.0-1.1 (/var/lib/apt/lists/pkgs.k8s.io_core:_stable:_v1.30_deb_Packages)
6 Description Language:
7   File:
8   /var/lib/apt/lists/pkgs.k8s.io_core:_stable:_v1.30_deb_Packages
9   MD5: dd712e8daa61f5a232c282fd36f21dc9
10 Description Language:
11   File:
12   /var/lib/apt/lists/pkgs.k8s.io_core:_stable:_v1.30_deb_Packages
13   MD5: dd712e8daa61f5a232c282fd36f21dc9
14 Description Language:
15   File:
16   /var/lib/apt/lists/pkgs.k8s.io_core:_stable:_v1.30_deb_Packages
17   MD5: dd712e8daa61f5a232c282fd36f21dc9
18
19 Reverse Depends:
20   kubeadm:arm64,kubeadm
21   kubeadm:ppc64el,kubeadm
22   kubeadm:s390x,kubeadm
23 Dependencies:
24 1.30.0-1.1 - cri-tools (2 1.30.0) kubeadm:arm64 (32 (null)) kubeadm:s390x
25 (32 (null)) kubeadm:ppc64el (32 (null))
26 Provides:
27 1.30.0-1.1 -
28 Reverse Provides:
```

```
1 查看可以软件列表
2 # apt-cache madison kubeadm
3   kubeadm | 1.30.0-1.1 | https://pkgs.k8s.io/core:/stable:/v1.30/deb
   Packages
```

```
1 默认安装
2 # sudo apt-get install -y kubelet kubeadm kubectl
```

```
1 安装指定版本
2 # sudo apt-get install -y kubelet=1.30.0-1.1 kubeadm=1.30.0-1.1
   kubectl=1.30.0-1.1
```

```
1 如有报错:
2 E: Could not get lock /var/lib/dpkg/lock-frontent. It is held by process 5005
   (unattended-upgr)
3 N: Be aware that removing the lock file is not a solution and may break your
   system.
4 E: Unable to acquire the dpkg frontend lock (/var/lib/dpkg/lock-frontent), is
   another process using it?
5 可稍等等
```

```
1 锁定版本，防止后期自动更新
2 # sudo apt-mark hold kubelet kubeadm kubectl
```

```
1 解锁版本，可以执行更新
2 # sudo apt-mark unhold kubelet kubeadm kubectl
```

3.2.2 配置kubelet

为了实现容器运行时使用的cgroupdriver与kubelet使用的cgroup的一致性，建议修改如下文件内容。

```
1 # vim /etc/sysconfig/kubelet
2 KUBELET_EXTRA_ARGS="--cgroup-driver=systemd"
```

```
1 设置kubelet为开机自启动即可，由于没有生成配置文件，集群初始化后自动启动
2 # systemctl enable kubelet
```

3.3 K8S集群初始化

3.3.1 查看版本

```
1 root@k8s-master01:~# kubeadm version
2 kubeadm version: &version.Info{Major:"1", Minor:"30", GitVersion:"v1.30.0",
  GitCommit:"7c48c2bd72b9bf5c44d21d7338cc7bea77d0ad2a", GitTreeState:"clean",
  BuildDate:"2024-04-17T17:34:08Z", GoVersion:"go1.22.2", Compiler:"gc",
  Platform:"linux/amd64"}
```

3.3.2 生成部署配置文件

```
1 root@k8s-master01:~# kubeadm config print init-defaults > kubeadm-config.yaml
```

使用kubernetes社区版容器镜像仓库

```
1 root@k8s-master01:~# vim kubeadm-config.yaml
2 root@k8s-master01:~# cat kubeadm-config.yaml
3 apiVersion: kubeadm.k8s.io/v1beta3
4 bootstrapTokens:
5   - groups:
6     - system:bootstrappers:kubeadm:default-node-token
7     token: abcdef.0123456789abcdef
8     ttl: 24h0m0s
9     usages:
10    - signing
11    - authentication
12 kind: InitConfiguration
13 localAPIEndpoint:
14   advertiseAddress: 192.168.10.140
15   bindPort: 6443
16 nodeRegistration:
17   criSocket: unix:///var/run/containerd/containerd.sock
18   imagePullPolicy: IfNotPresent
19   name: k8s-master01
20   taints: null
21 ---
22 apiServer:
23   timeoutForControlPlane: 4m0s
24 apiVersion: kubeadm.k8s.io/v1beta3
25 certificatesDir: /etc/kubernetes/pki
26 clusterName: kubernetes
27 controllerManager: {}
28 dns: {}
29 etcd:
30   local:
31     dataDir: /var/lib/etcd
32 imageRepository: registry.k8s.io
33 kind: ClusterConfiguration
34 kubernetesVersion: 1.30.0
35 networking:
36   dnsDomain: cluster.local
```

```
37     servicesSubnet: 10.96.0.0/12
38     podSubnet: 10.244.0.0/16
39     scheduler: {}
40     ---
41     kind: KubeletConfiguration
42     apiVersion: kubelet.config.k8s.io/v1beta1
43     cgroupDriver: systemd
```

使用阿里云容器镜像仓库

```
1 root@k8s-master01:~# vim kubeadm-config.yaml
2 root@k8s-master01:~# cat kubeadm-config.yaml
3 apiVersion: kubeadm.k8s.io/v1beta3
4 bootstrapTokens:
5   - groups:
6     - system:bootstrappers:kubeadm:default-node-token
7     token: abcdef.0123456789abcdef
8     ttl: 24h0m0s
9     usages:
10    - signing
11    - authentication
12 kind: InitConfiguration
13 localAPIEndpoint:
14   advertiseAddress: 192.168.10.140
15   bindPort: 6443
16 nodeRegistration:
17   criSocket: unix:///var/run/containerd/containerd.sock
18   imagePullPolicy: IfNotPresent
19   name: k8s-master01
20   taints: null
21   ---
22 apiServer:
23   timeoutForControlPlane: 4m0s
24 apiVersion: kubeadm.k8s.io/v1beta3
25 certificatesDir: /etc/kubernetes/pki
26 clusterName: kubernetes
27 controllerManager: {}
28 dns: {}
29 etcd:
30   local:
31     dataDir: /var/lib/etcd
32 imageRepository: registry.aliyuncs.com/google_containers
33 kind: ClusterConfiguration
34 kubernetesVersion: 1.30.0
35 networking:
36   dnsDomain: cluster.local
37   servicesSubnet: 10.96.0.0/12
38   podSubnet: 10.244.0.0/16
39   scheduler: {}
40   ---
41   kind: KubeletConfiguration
42   apiVersion: kubelet.config.k8s.io/v1beta1
43   cgroupDriver: systemd
```

3.3.3 查看并下载镜像

```
1 root@k8s-master01:~# kubeadm config images list
2
3 registry.k8s.io/kube-apiserver:v1.30.0
4 registry.k8s.io/kube-controller-manager:v1.30.0
5 registry.k8s.io/kube-scheduler:v1.30.0
6 registry.k8s.io/kube-proxy:v1.30.0
7 registry.k8s.io/coredns/coredns:v1.11.1
8 registry.k8s.io/pause:3.9
9 registry.k8s.io/etcd:3.5.12-0
```

```
1 使用阿里云容器镜像仓库
2 # kubeadm config images list --image-repository
  registry.aliyuncs.com/google_containers
3 registry.aliyuncs.com/google_containers/kube-apiserver:v1.30.0
4 registry.aliyuncs.com/google_containers/kube-controller-manager:v1.30.0
5 registry.aliyuncs.com/google_containers/kube-scheduler:v1.30.0
6 registry.aliyuncs.com/google_containers/kube-proxy:v1.30.0
7 registry.aliyuncs.com/google_containers/coredns:v1.11.1
8 registry.aliyuncs.com/google_containers/pause:3.9
9 registry.aliyuncs.com/google_containers/etcd:3.5.12-0
```

```
1 root@k8s-master01:~# kubeadm config images pull
2
3 [config/images] Pulled registry.k8s.io/kube-apiserver:v1.30.0
4 [config/images] Pulled registry.k8s.io/kube-controller-manager:v1.30.0
5 [config/images] Pulled registry.k8s.io/kube-scheduler:v1.30.0
6 [config/images] Pulled registry.k8s.io/kube-proxy:v1.30.0
7 [config/images] Pulled registry.k8s.io/coredns/coredns:v1.11.1
8 [config/images] Pulled registry.k8s.io/pause:3.9
9 [config/images] Pulled registry.k8s.io/etcd:3.5.12-0
```

```
1 使用阿里云容器镜像仓库
2 # kubeadm config images pull --image-repository
  registry.aliyuncs.com/google_containers
3 [config/images] Pulled registry.aliyuncs.com/google_containers/kube-
  apiserver:v1.30.0
4 [config/images] Pulled registry.aliyuncs.com/google_containers/kube-
  controller-manager:v1.30.0
5 [config/images] Pulled registry.aliyuncs.com/google_containers/kube-
  scheduler:v1.30.0
6 [config/images] Pulled registry.aliyuncs.com/google_containers/kube-
  proxy:v1.30.0
7 [config/images] Pulled
  registry.aliyuncs.com/google_containers/coredns:v1.11.1
8 [config/images] Pulled registry.aliyuncs.com/google_containers/pause:3.9
9 [config/images] Pulled registry.aliyuncs.com/google_containers/etcd:3.5.12-0
```

3.3.4 使用部署配置文件初始化K8S集群

```
1 root@k8s-master01:~# kubeadm init --config kubeadm-config.yaml
```

```
1 输出内容如下:
2
3 [init] Using Kubernetes version: v1.30.0
4 [preflight] Running pre-flight checks
5 [preflight] Pulling images required for setting up a Kubernetes cluster
6 [preflight] This might take a minute or two, depending on the speed of your
  internet connection
7 [preflight] You can also perform this action in beforehand using 'kubeadm
  config images pull'
8 [certs] Using certificateDir folder "/etc/kubernetes/pki"
9 [certs] Generating "ca" certificate and key
10 [certs] Generating "apiserver" certificate and key
11 [certs] apiserver serving cert is signed for DNS names [k8s-master01
  kubernetes kubernetes.default kubernetes.default.svc
  kubernetes.default.svc.cluster.local] and IPs [10.96.0.1 192.168.10.140]
12 [certs] Generating "apiserver-kubelet-client" certificate and key
13 [certs] Generating "front-proxy-ca" certificate and key
14 [certs] Generating "front-proxy-client" certificate and key
15 [certs] Generating "etcd/ca" certificate and key
16 [certs] Generating "etcd/server" certificate and key
17 [certs] etcd/server serving cert is signed for DNS names [k8s-master01
  localhost] and IPs [192.168.10.140 127.0.0.1 ::1]
18 [certs] Generating "etcd/peer" certificate and key
19 [certs] etcd/peer serving cert is signed for DNS names [k8s-master01
  localhost] and IPs [192.168.10.140 127.0.0.1 ::1]
20 [certs] Generating "etcd/healthcheck-client" certificate and key
21 [certs] Generating "apiserver-etcd-client" certificate and key
22 [certs] Generating "sa" key and public key
23 [kubeconfig] Using kubeconfig folder "/etc/kubernetes"
24 [kubeconfig] Writing "admin.conf" kubeconfig file
```

```
25 [kubeconfig] writing "super-admin.conf" kubeconfig file
26 [kubeconfig] writing "kubelet.conf" kubeconfig file
27 [kubeconfig] writing "controller-manager.conf" kubeconfig file
28 [kubeconfig] writing "scheduler.conf" kubeconfig file
29 [etcd] Creating static Pod manifest for local etcd in
   "/etc/kubernetes/manifests"
30 [control-plane] Using manifest folder "/etc/kubernetes/manifests"
31 [control-plane] Creating static Pod manifest for "kube-apiserver"
32 [control-plane] Creating static Pod manifest for "kube-controller-manager"
33 [control-plane] Creating static Pod manifest for "kube-scheduler"
34 [kubelet-start] Writing kubelet environment file with flags to file
   "/var/lib/kubelet/kubeadm-flags.env"
35 [kubelet-start] Writing kubelet configuration to file
   "/var/lib/kubelet/config.yaml"
36 [kubelet-start] Starting the kubelet
37 [wait-control-plane] waiting for the kubelet to boot up the control plane as
   static Pods from directory "/etc/kubernetes/manifests"
38 [kubelet-check] waiting for a healthy kubelet. This can take up to 4m0s
39 [kubelet-check] The kubelet is healthy after 501.627385ms
40 [api-check] waiting for a healthy API server. This can take up to 4m0s
41 [api-check] The API server is healthy after 4.502636534s
42 [upload-config] Storing the configuration used in ConfigMap "kubeadm-config"
   in the "kube-system" Namespace
43 [kubelet] Creating a ConfigMap "kubelet-config" in namespace kube-system
   with the configuration for the kubelets in the cluster
44 [upload-certs] Skipping phase. Please see --upload-certs
45 [mark-control-plane] Marking the node k8s-master01 as control-plane by
   adding the labels: [node-role.kubernetes.io/control-plane
   node.kubernetes.io/exclude-from-external-load-balancers]
46 [mark-control-plane] Marking the node k8s-master01 as control-plane by
   adding the taints [node-role.kubernetes.io/control-plane:Noschedule]
47 [bootstrap-token] Using token: abcdef.0123456789abcdef
48 [bootstrap-token] Configuring bootstrap tokens, cluster-info ConfigMap, RBAC
   Roles
49 [bootstrap-token] Configured RBAC rules to allow Node Bootstrap tokens to
   get nodes
50 [bootstrap-token] Configured RBAC rules to allow Node Bootstrap tokens to
   post CSRs in order for nodes to get long term certificate credentials
51 [bootstrap-token] Configured RBAC rules to allow the csrapprover controller
   automatically approve CSRs from a Node Bootstrap Token
52 [bootstrap-token] Configured RBAC rules to allow certificate rotation for
   all node client certificates in the cluster
53 [bootstrap-token] Creating the "cluster-info" ConfigMap in the "kube-public"
   namespace
54 [kubelet-finalize] Updating "/etc/kubernetes/kubelet.conf" to point to a
   rotatable kubelet client certificate and key
55 [addons] Applied essential addon: CoreDNS
56 [addons] Applied essential addon: kube-proxy
57
58 Your Kubernetes control-plane has initialized successfully!
59
60 To start using your cluster, you need to run the following as a regular
   user:
61
62     mkdir -p $HOME/.kube
63     sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
64     sudo chown $(id -u):$(id -g) $HOME/.kube/config
65
```



```
66 Alternatively, if you are the root user, you can run:
67
68     export KUBECONFIG=/etc/kubernetes/admin.conf
69
70 You should now deploy a pod network to the cluster.
71 Run "kubectl apply -f [podnetwork].yaml" with one of the options listed at:
72     https://kubernetes.io/docs/concepts/cluster-administration/addons/
73
74 Then you can join any number of worker nodes by running the following on
    each as root:
75
76 kubectl join 192.168.10.140:6443 --token abcdef.0123456789abcdef \
77     --discovery-token-ca-cert-hash
    sha256:064c6d804cbda71938e8aeb3f46485c99fc90d08b58f05a317e5c78903a62814
```

3.4 准备kubectl配置文件

仅在k8s-master节点上进行操作。

```
1 mkdir -p $HOME/.kube
2 sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
3 sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

3.5 工作节点加入集群

```
1 root@k8s-worker01:~# kubectl join 192.168.10.140:6443 --token
    abcdef.0123456789abcdef \
2     --discovery-token-ca-cert-hash
    sha256:9dcd58f0081f4fed20fa2ecc5343b722376eaff556f0d2fdf43b1924db02a48
```

```
1 root@k8s-worker02:~# kubectl join 192.168.10.140:6443 --token
    abcdef.0123456789abcdef \
2     --discovery-token-ca-cert-hash
    sha256:9dcd58f0081f4fed20fa2ecc5343b722376eaff556f0d2fdf43b1924db02a48
```

3.6 验证K8S集群节点是否可用

```

1 root@k8s-master01:~# kubectl get nodes
2 NAME                STATUS    ROLES    AGE     VERSION
3 k8s-master01        NotReady control-plane 7m28s   v1.30.0
4 k8s-worker01        NotReady <none>    29s     v1.30.0
5 k8s-worker02        NotReady <none>    24s     v1.30.0

```

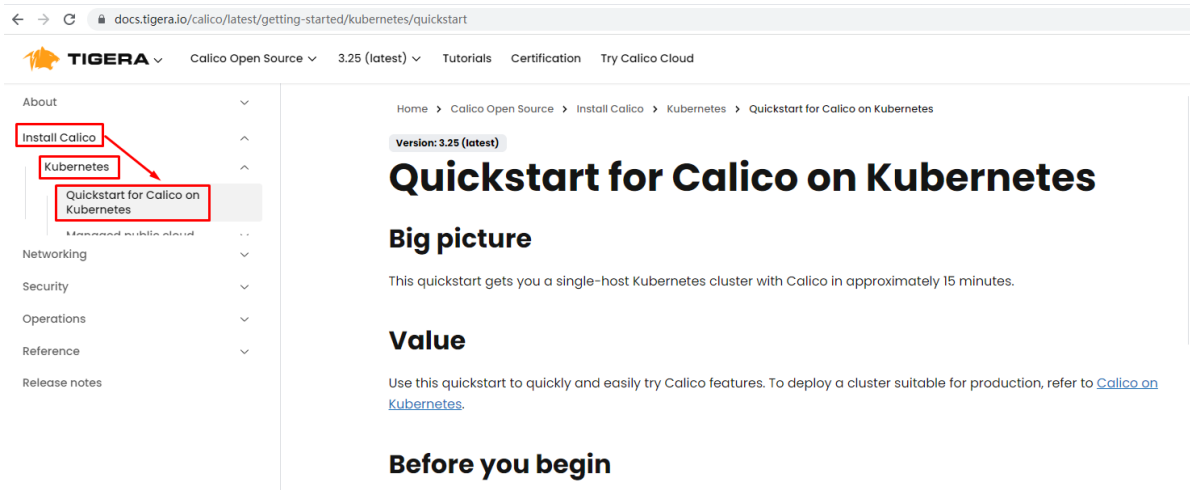
```

1 root@k8s-master01:~# kubectl get pods -n kube-system
2 NAME                                READY   STATUS    RESTARTS   AGE
3 NAME                                READY   STATUS    RESTARTS   AGE
4 coredns-7db6d8ff4d-8b57k           0/1     Pending   0           7m54s
5 coredns-7db6d8ff4d-8k6nr           0/1     Pending   0           7m54s
6 etcd-k8s-master01                  1/1     Running   0           8m8s
7 kube-apiserver-k8s-master01         1/1     Running   0           8m8s
8 kube-controller-manager-k8s-master01 1/1     Running   0           8m8s
9 kube-proxy-5tn6g                    1/1     Running   0           67s
10 kube-proxy-nv5nq                    1/1     Running   0           72s
11 kube-proxy-w2xj6                    1/1     Running   0           7m54s
12 kube-scheduler-k8s-master01         1/1     Running   0           8m9s

```

四、K8S集群网络插件calico部署

calico访问链接：<https://projectcalico.docs.tigera.io/about/about-calico>



docs.tigera.io/calico/latest/getting-started/kubernetes/quickstart

TIGERA Calico Open Source 3.25 (latest) Tutorials Certification Try Calico Cloud

About

- Install Calico
- Kubernetes
- Quickstart for Calico on Kubernetes
- Managed public cloud
- Networking
- Security
- Operations
- Reference
- Release notes

Home > Calico Open Source > Install Calico > Kubernetes > Quickstart for Calico on Kubernetes

Version: 3.25 (latest)

Quickstart for Calico on Kubernetes

Big picture

This quickstart gets you a single-host Kubernetes cluster with Calico in approximately 15 minutes.

Value

Use this quickstart to quickly and easily try Calico features. To deploy a cluster suitable for production, refer to [Calico on Kubernetes](#).

Before you begin

docs.tigera.io/calico/latest/getting-started/kubernetes/quickstart

TIGERA Calico Open Source 3.26 (latest) Tutorials Certification Try Calico Cloud Contact us

About
Install Calico
Kubernetes
System requirements
Community-tested Kubernetes versions
Quickstart for Calico on Kubernetes
Managed public cloud
Self-managed public cloud
Self-managed on-premises
OpenShift
Rancher Kubernetes Engine (RKE)
Flannel
Calico for Windows
K3s
Install using Helm
Quickstart for Calico on MicroK8s
Quickstart for Calico on minikube
Kind multi-node install
Calico the hard way

NOTE
If 192.168.0.0/16 is already in use within your network you must select a different pod network CIDR, replacing 192.168.0.0/16 in the above command.

4. Execute the following commands to configure kubectl (also returned by `kubeadm init`).

```
mkdir -p $HOME/.kube  
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config  
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

Install Calico

1. Install the Tigera Calico operator and custom resource definitions. 第一步：复制后直接在K8S集群主机节点上执行即可。

```
kubectl create -f https://raw.githubusercontent.com/projectcalico/calico/v3.26.1/manifests/tigera-operator.yaml
```

NOTE
Due to the large size of the CRD bundle, `kubectl apply` might exceed request limits. Instead, use `kubectl create` or `kubectl replace`.

2. Install Calico by creating the necessary custom resource. For more information on configuration options available in this manifest, see [the installation reference](#). 第二步：先使用wget下载，然后修改其中关于pod网络后再使用kubectl create执行即可。

```
kubectl create -f https://raw.githubusercontent.com/projectcalico/calico/v3.26.1/manifests/custom-resources.yaml
```

```
1 root@k8s-master01:~# kubectl create -f  
https://raw.githubusercontent.com/projectcalico/calico/v3.26.1/manifests/tigera-operator.yaml
```

```
1 输出内容:  
2 namespace/tigera-operator created  
3 customresourcedefinition.apiextensions.k8s.io/bgpconfigurations.crd.projectcalico.org created  
4 customresourcedefinition.apiextensions.k8s.io/bgpfilters.crd.projectcalico.org created  
5 customresourcedefinition.apiextensions.k8s.io/bgppeers.crd.projectcalico.org created  
6 customresourcedefinition.apiextensions.k8s.io/blockaffinities.crd.projectcalico.org created  
7 customresourcedefinition.apiextensions.k8s.io/caliconodestatuses.crd.projectcalico.org created  
8 customresourcedefinition.apiextensions.k8s.io/clusterinformations.crd.projectcalico.org created  
9 customresourcedefinition.apiextensions.k8s.io/felixconfigurations.crd.projectcalico.org created  
10 customresourcedefinition.apiextensions.k8s.io/globalnetworkpolicies.crd.projectcalico.org created  
11 customresourcedefinition.apiextensions.k8s.io/globalnetworksets.crd.projectcalico.org created  
12 customresourcedefinition.apiextensions.k8s.io/hostendpoints.crd.projectcalico.org created  
13 customresourcedefinition.apiextensions.k8s.io/ipamblocks.crd.projectcalico.org created  
14 customresourcedefinition.apiextensions.k8s.io/ipamconfigs.crd.projectcalico.org created  
15 customresourcedefinition.apiextensions.k8s.io/ipamhandles.crd.projectcalico.org created  
16 customresourcedefinition.apiextensions.k8s.io/ippools.crd.projectcalico.org created
```

```
17 customresourcedefinition.apiextensions.k8s.io/ipreservations.crd.projectcali
   co.org created
18 customresourcedefinition.apiextensions.k8s.io/kubecontrollersconfigurations.
   crd.projectcalico.org created
19 customresourcedefinition.apiextensions.k8s.io/networkpolicies.crd.projectcal
   ico.org created
20 customresourcedefinition.apiextensions.k8s.io/networksets.crd.projectcalico.
   org created
21 customresourcedefinition.apiextensions.k8s.io/apiservers.operator.tigera.io
   created
22 customresourcedefinition.apiextensions.k8s.io/imagesets.operator.tigera.io
   created
23 customresourcedefinition.apiextensions.k8s.io/installations.operator.tigera.
   io created
24 customresourcedefinition.apiextensions.k8s.io/tigerastatuses.operator.tigera
   .io created
25 serviceaccount/tigera-operator created
26 clusterrole.rbac.authorization.k8s.io/tigera-operator created
27 clusterrolebinding.rbac.authorization.k8s.io/tigera-operator created
28 deployment.apps/tigera-operator created
```

```
1 # wget
   https://raw.githubusercontent.com/projectcalico/calico/v3.26.1/manifests/cust
   om-resources.yaml
```

```
1 # vim custom-resources.yaml
2
3
4 # cat custom-resources.yaml
5
6
7 # This section includes base Calico installation configuration.
8 # For more information, see:
   https://projectcalico.docs.tigera.io/master/reference/installation/api#opera
   tor.tigera.io/v1.Installation
9 apiVersion: operator.tigera.io/v1
10 kind: Installation
11 metadata:
12   name: default
13 spec:
14   # Configures Calico networking.
15   calicoNetwork:
16     # Note: The ipPools section cannot be modified post-install.
17     ipPools:
18     - blockSize: 26
19       cidr: 10.244.0.0/16 修改此行为初始化时定义的pod network cidr
20       encapsulation: VXLANCrossSubnet
21       natOutgoing: Enabled
22       nodeSelector: all()
23
24 ---
25
```

```

26 # This section configures the Calico API server.
27 # For more information, see:
    https://projectcalico.docs.tigera.io/master/reference/installation/api#opera
    tor.tigera.io/v1.APIServer
28 apiVersion: operator.tigera.io/v1
29 kind: APIServer
30 metadata:
31   name: default
32 spec: {}

```

```

1 # kubectl create -f custom-resources.yaml
2
3 installation.operator.tigera.io/default created
4 apiserver.operator.tigera.io/default created

```

```

1 root@k8s-master01:~# kubectl get pods -n calico-system
2 NAME                                                    READY   STATUS    RESTARTS   AGE
3 calico-kube-controllers-76bbb9b96b-rvdrd              1/1     Running   0           15m
4 calico-node-cp5xf                                       1/1     Running   0           15m
5 calico-node-tv27t                                       1/1     Running   0           15m
6 calico-node-x2c4p                                       1/1     Running   0           15m
7 calico-typha-65c8d59447-ldfbp                         1/1     Running   0           14m
8 calico-typha-65c8d59447-zlcjt                         1/1     Running   0           15m
9 csi-node-driver-2zr9w                                   2/2     Running   0           15m
10 csi-node-driver-5zvzq                                   2/2     Running   0           15m
11 csi-node-driver-bs5b2                                   2/2     Running   0           15m

```

五、部署Nginx应用验证K8S集群可用性

```

1 root@k8s-master01:~# vim nginx.yaml
2 root@k8s-master01:~# cat nginx.yaml
3 ---
4 apiVersion: apps/v1
5 kind: Deployment
6 metadata:
7   name: nginxweb
8 spec:
9   selector:
10     matchLabels:
11       app: nginxweb1
12   replicas: 2
13   template:
14     metadata:
15       labels:
16         app: nginxweb1
17   spec:
18     containers:

```

```
19     - name: nginxwebc
20       image: nginx:latest
21       imagePullPolicy: IfNotPresent
22       ports:
23     - containerPort: 80
24
25 ---
26 apiVersion: v1
27 kind: Service
28 metadata:
29   name: nginxweb-service
30 spec:
31   externalTrafficPolicy: Cluster
32   selector:
33     app: nginxweb1
34   ports:
35   - protocol: TCP
36     port: 80
37     targetPort: 80
38     nodePort: 30080
39   type: NodePort
```

```
1 root@k8s-master01:~# kubectl apply -f nginx.yaml
2 deployment.apps/nginxweb created
3 service/nginxweb-service created
```

Welcome to nginx! x +
← → ↻ ⚠ 不安全 | 192.168.10.140:30080

Welcome to nginx!

If you see this page, the nginx web server is successfully installed and working. Further configuration is required.

For online documentation and support please refer to nginx.org.
Commercial support is available at nginx.com.

Thank you for using nginx.